Métodos Numéricos para Sistemas de Ecuaciones Lineales

Este cuaderno reúne los códigos proporcionados para los cuatro métodos en el siguiente orden:

1. Eliminación de Gauss
2. Descomposición LU
3. Método de Jacobi
4. Método de Gauss-Seidel

Los códigos se organizan en un único archivo para facilitar su uso en clase.

1. Eliminación de Gauss

```
import numpy as np

def gaussian_elimination(A, b):
    # Convert inputs to float arrays
    A = np.array(A, dtype=float)
    b = np.array(b, dtype=float)
    n = len(b)

    Ab = np.hstack([A, b.reshape(-1, 1)])

    for k in range(n - 1):
        for i in range(k + 1, n):
            factor = Ab[i, k] / Ab[k, k]
            for j in range(k, n + 1):
                Ab[i, j] -= factor * Ab[k, j]

    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        sum_ax = 0.0
        for j in range(i + 1, n):
           sum_ax += Ab[i, j] * x[j]
        x[i] = (Ab[i, n] - sum_ax) / Ab[i, i]

    return x

A = [[2, 1, -1],
     [-3, -1, 2],
     [-2, 1, 2]]
b = [8, -11, -3]

solution = gaussian_elimination(A, b)
print("Solution x:", solution)
```

2. Descomposición LU

Implementación de LU

```
import numpy as np
from numpy import linalg
```

```
def lu(A):

    n=A.shape[0]
```

```
    U=A.copy().astype(np.double)

    L=np.eye(n,dtype=np.double)

    for i in range(n-1):

        factor = U[i+1:, i] / U[i, i]
        L[i+1:, i] = factor
        U[i+1:] -= factor[:, np.newaxis] * U[i]


    return L,U
```

Implementación de PLU

```
def plu(A):

    n = A.shape[0]

    U = A.copy().astype(np.double)
    L = np.eye(n, dtype=np.double)
    P = np.eye(n, dtype=np.double)



#Permutar U para obtener los valores indicados para la diagonal
    for i in range(n-1):
        pivote=np.argmax(np.abs(U[i:,i]))+i

        if pivote != i:
            U[[i, pivote]] = U[[pivote, i]]
            P[[i, pivote]] = P[[pivote, i]]
            if i > 0:
                L[[i, pivote], :i] = L[[pivote, i], :i]


        factor = U[i+1:, i] / U[i, i]
        L[i+1:, i] = factor
        U[i+1:] -= factor[:, np.newaxis] * U[i]

    return P, L, U
```

Sustitución hacia adelante y hacia atrás

```
def adelante(L,b):

    n=L.shape[0]
    y=np.zeros_like(b,dtype=np.double)

    for i in range(n):

        y[i]=(b[i]-np.dot(L[i,:i],y[:i]))/L[i,i]

    return y


def atras(U,y):

    n=U.shape[0]
    x=np.zeros_like(y,dtype=np.double)

    for i in range(n - 1, -1, -1):

        x[i]=(y[i]-np.dot(U[i,i:],x[i:]))/U[i,i]

    return x
```

Solución de sistemas

```
def solucion_lu(A,b):

    L,U=lu(A)


    y=adelante(L,b)

    return atras(U,y)

def solucion_plu(A,b):

    P,L,U=plu(A)

    Pb=np.dot(P,b)

    y=adelante(L,Pb)

    return atras(U,y)
```

Ejemplos

```
# 1.

A1 = np.array([
    [ 2.0,  1.0, -1.0],
    [-3.0, -1.0,  2.0],
    [-2.0,  1.0,  2.0]
])

b1 = np.array([8.0, -11.0, -3.0])
```

```
P1, L1, U1 = plu(A1)
print(P1)
print(L1)
print(U1)
y1=adelante(L1,b1)
x1=atras(U1,y1)
solucion_plu(A1,b1)
```

```
# 2.

A2=np.array([
    [0.0,1.0],
    [1.0,1.0]
])

b2=np.array([2.0,1.0])

L2,U2=lu(A2)
y2=adelante(L2,b2)
x2=atras(U2,y2)
solucion_lu(A2,b2)
```

```
A2=np.array([
    [0.0,1.0],
    [1.0,1.0]
])

b2=np.array([2.0,1.0])

P2,L2,U2=plu(A2)
```

```
y2=adelante(L2,b2)
x2=atras(U2,y2)
solucion_plu(A2,b2)
```

```
# 3.

A3=np.array([
    [4.0,3.0,0.0,-1.0],
    [5.0,4.0,3.0,6.0],
    [-3.0,0.0,4.0,2.0],
    [-4.0,2.0,3.0,4.0]
])
b3=np.array([5.0,2.0,3.0,1.0])
P3,L3,U3=plu(A3)
y3=adelante(L3,b3)
x3=atras(U3,y3)
solucion_plu(A3,b3)
```

3. Método de Jacobi

Funciones auxiliares y criterios de convergencia

```
import numpy as np
metodos=("jacobi","gauss_seidel")

# Criterio suficiente pero no necesario de convergencia: diagonal dominante por filas
def convergencia_suficiente(A:np.array):
    n=A.shape[0]
    for i in range(n):
        suma=np.sum(np.abs(A[i,:]))-np.abs(A[i,i])
        if np.abs(A[i,i])<=suma:
            return False
    return True

# Criterio necesario de convergencia: diagonal mayor por filas
def convergencia_necesaria(A:np.array):
    n=A.shape[0]
    for i in range(n):
        for j in range(n):
            if i!=j and np.abs(A[i,j])>np.abs(A[i,i]):
                return False
    return True

# Arreglar matriz teniendo en cuenta que el criterio de convergencia necesario pero no suficiente
def arreglar(A:np.array,b:np.array):
    n=A.shape[0]
    for i in range(n):
        for j in range(n):
            if i!=j and np.abs(A[i,j])>np.abs(A[i,i]):
                A[[i,j]]=A[[j,i]]
                b[[i,j]]=b[[j,i]]
    return A,b

# Criterio de convergencia por radio espectral
def convergencia_rho(A:np.array,metodo:str):
    def radio_espectral(A:np.array,metodo:str):
        n=A.shape[0]
        if metodo=="jacobi":
            D=np.diag(np.diag(A))
            R=A-D
            T=-np.linalg.inv(D)@R
        elif metodo=="gauss_seidel":
            D=np.diag(np.diag(A))
            L=np.tril(A)-D
            U=np.triu(A)-D
            T=np.linalg.inv(D+L)@U
        else:
            raise ValueError("Use jacobi o gauss_seidel")
```

```
        eigenv,_=np.linalg.eig(T)
        return max(np.abs(eigenv))
    return radio_espectral(A,metodo)<1
```

```
# Método de Jacobi (itera sobre x_old)
def jacobi(A:np.array,b:np.array,tol:float,max_iter:int):
    n=A.shape[0]
    x0=np.zeros(n)
    x=x0.copy()
    for k in range(max_iter):
        x_old=x.copy()
        for i in range(n):
            suma=np.dot(A[i,:],x_old)-A[i,i]*x_old[i]
            x[i]=(b[i]-suma)/A[i,i]
        if np.linalg.norm(x-x_old,ord=np.inf)<tol:
            return x
    return x
```

4. Método de Gauss-Seidel

```
# Método de Gauss-Seidel (itera sobre x)
def gauss_seidel(A:np.array,b:np.array,tol:float,max_iter:int):
    n=A.shape[0]
    x0=np.zeros(n)
    x=x0.copy()
    for k in range(max_iter):
        x_old=x.copy()
        for i in range(n):
            suma=np.dot(A[i,:],x)-A[i,i]*x[i]
            x[i]=(b[i]-suma)/A[i,i]
        if np.linalg.norm(x-x_old,ord=np.inf)<tol:
            return x
    return x
```

Ejemplos de Jacobi y Gauss-Seidel

```
# Ejemplo 1
A1=np.array([
    [4,-1,0,0,0,0,0,0],
    [-1,4,-1,0,0,0,0,0],
    [0,-1,4,-1,0,0,0,0],
    [0,0,-1,4,-1,0,0,0],
    [0,0,0,-1,4,-1,0,0],
    [0,0,0,0,-1,4,-1,0],
    [0,0,0,0,0,-1,4,-1],
    [0,0,0,0,0,0,-1,4],])
b1=np.array([1,2,3,4,5,6,7,8])

for metodo in metodos:
    print(f"Método: {metodo}")
    if convergencia_suficiente(A1): print("Criterio suficiente de convergencia: Sí")
    else: print("Criterio suficiente de convergencia: No")
    if convergencia_necesaria(A1): print("Criterio necesario de convergencia: Sí")
    else: print("Criterio necesario de convergencia: No")
    if convergencia_rho(A1,metodo):
        print("Criterio de convergencia por radio espectral: Sí")
        if metodo=="jacobi": x=jacobi(A1,b1,tol=1e-6,max_iter=100)
        else: x=gauss_seidel(A1,b1,tol=1e-6,max_iter=100)
        print(f"Solución aproximada: {x}\n",
              f"Confirmación: {A1@x}\n")
    else: print("Criterio de convergencia por radio espectral: No\n")
```

```
# Ejemplo 2
A2=np.array([
    [1,-4,0,0,0,0,0,0],
    [-4,1,-4,0,0,0,0,0],
    [0,-4,1,-4,0,0,0,0],
    [0,0,-4,1,-4,0,0,0],
    [0,0,0,-4,1,-4,0,0],
    [0,0,0,0,-4,1,-4,0],
    [0,0,0,0,0,-4,1,-4],
    [0,0,0,0,0,0,-4,1],])
b2=np.array([8,7,6,5,4,3,2,1])

for metodo in metodos:
    print(f"Método: {metodo}")
    if convergencia_suficiente(A2): print("Criterio suficiente de convergencia: Sí")
    else: print("Criterio suficiente de convergencia: No")
    if convergencia_necesaria(A2): print("Criterio necesario de convergencia: Sí")
    else: print("Criterio necesario de convergencia: No")
    if convergencia_rho(A2,metodo):
        print("Criterio de convergencia por radio espectral: Sí")
        if metodo=="jacobi": x=jacobi(A2,b2,tol=1e-6,max_iter=100)
        else: x=gauss_seidel(A2,b2,tol=1e-6,max_iter=100)
        print(f"Solución aproximada: {x}\n",
              f"Confirmación: {A2@x}\n")
    else: print("Criterio de convergencia por radio espectral: No\n")
```

```
A2=arreglar(A2,b2)[0]
b2=arreglar(A2,b2)[1]
for metodo in metodos:
    print(f"Método: {metodo}")
    if convergencia_suficiente(A2): print("Criterio suficiente de convergencia: Sí")
    else: print("Criterio suficiente de convergencia: No")
    if convergencia_necesaria(A2): print("Criterio necesario de convergencia: Sí")
    else: print("Criterio necesario de convergencia: No")
    if convergencia_rho(A2,metodo):
        print("Criterio de convergencia por radio espectral: Sí")
        if metodo=="jacobi": x=jacobi(A2,b2,tol=1e-6,max_iter=100)
        else: x=gauss_seidel(A2,b2,tol=1e-6,max_iter=100)
        print(f"Solución aproximada: {x}\n",
              f"Confirmación: {A2@x}\n")
    else: print("Criterio de convergencia por radio espectral: No\n")
```

```
# Ejemplo 3
A3=np.array([
    [5,-4,0,0,0,0,0,0],
    [-4,5,-4,0,0,0,0,0],
    [0,-4,5,-4,0,0,0,0],
    [0,0,-4,5,-4,0,0,0],
    [0,0,0,-4,5,-4,0,0],
    [0,0,0,0,-4,5,-4,0],
    [0,0,0,0,0,-4,5,-4],
    [0,0,0,0,0,0,-4,5],])
b3=np.array([8,7,6,5,4,3,2,1])

for metodo in metodos:
    print(f"Método: {metodo}")
    if convergencia_suficiente(A3): print("Criterio suficiente de convergencia: Sí")
    else: print("Criterio suficiente de convergencia: No")
    if convergencia_necesaria(A3): print("Criterio necesario de convergencia: Sí")
    else: print("Criterio necesario de convergencia: No")
    if convergencia_rho(A3,metodo):
        print("Criterio de convergencia por radio espectral: Sí")
        if metodo=="jacobi": x=jacobi(A3,b3,tol=1e-6,max_iter=100)
        else: x=gauss_seidel(A3,b3,tol=1e-6,max_iter=100)
        print(f"Solución aproximada: {x}\n",
              f"Confirmación: {A3@x}\n")
    else: print("Criterio de convergencia por radio espectral: No\n")
```